Níveis de reuso da SmartScale segundo o MPS.BR

Desenvolvimento Orientado a Reuso de Software · PUCPR · Engenharia de Software, 6º período · prof. Tiago Navarro

Referência: **MPS.BR — Guia Geral MPS de Software:2016**, processo Gerência de Reutilização (GRU, nível E, p. 36) e processo Desenvolvimento para Reutilização (DRU, nível C, p. 42).

Legenda:

- **Verde:** de acordo com o resultado esperado;
- **Amarelo:** identificação parcial;
- **Vermelho:** não encontrado.

O que é ativo reutilizável neste projeto:

- o componente `smartscale-core` 1.0.0, que contém o motor de rodízio, os contratos e o catálogo de critérios;
- o modelo de características;
- os padrões de projeto documentados;
- os templates do gerador Database First.

1. Gerência de Reutilização (GRU)

> *Propósito: gerenciar o ciclo de vida dos ativos reutilizáveis.*

1.1 Classificação

Resultado esperado	Situação	Justificativa (evidência no projeto)
GRU 1. Uma estratégia de gerenciamento de ativos é documentada, com a definição de ativo reutilizável e os critérios de aceitação, certificação, classificação, descontinuidade e avaliação	**Amarelo**	O que é o ativo e como usá-lo está documentado: README do `smartscale-core` (API pública, dependências, exemplo), `docs/VARIABILIDADE.md` (núcleo × produto) e o CHANGELOG com SemVer. **Falta** um documento de estratégia com os critérios de aceitação, certificação, classificação, descontinuidade e avaliação
GRU 2. Um mecanismo de armazenamento e recuperação de ativos reutilizáveis é implantado	**Amarelo**	O componente é empacotado em JAR e instalado no repositório Maven local (`mvn install` → `~/.m2/.../smartscale-core/1.0.0/`); o código-fonte fica no GitHub. **Falta** um repositório remoto de artefatos: o próprio README diz que o componente "ainda não está publicado num repositório remoto", então outro computador só obtém o ativo recompilando
GRU 3. Os dados de utilização dos ativos reutilizáveis são registrados	**Vermelho**	Não há registro de quem usa o ativo, em qual versão, desde quando. A única consumidora é o backend do MilScale, visível apenas lendo o `backend/pom.xml`
GRU 4. Os ativos reutilizáveis são periodicamente mantidos, segundo os	**Amarelo**	O controle de modificações existe: Git com branches e merge na `main`,

Resultado esperado	Situação	Justificativa (evidência no projeto)
critérios definidos, e suas modificações são controladas ao longo do ciclo de vida		versionamento semântico, CHANGELOG obrigatório, 7 testes do núcleo que rodam a cada build. **Falta** a manutenção periódica com critérios definidos: não há calendário de revisão nem integração contínua no repositório
GRU 5. Os usuários de ativos reutilizáveis são notificados sobre problemas, modificações, novas versões e descontinuidade	**Amarelo**	O CHANGELOG viaja dentro do JAR (`META-INF/CHANGELOG.md`) e registra cada versão, então quem atualiza fica sabendo o que mudou. **Falta** notificação ativa: ninguém é avisado de uma versão nova, de um defeito conhecido ou de uma descontinuidade

1.2 Como adequar o protótipo à GRU

Resultado	Proposta	Artefato ou ferramenta	Como saber que atende
GRU 1	Escrever a **estratégia de gerenciamento de ativos**. **Definição:** ativo é todo artefato versionado e documentado feito para ser usado por mais de um produto (componente de código, template de geração, modelo de domínio, documento de padrão). **Aceitação:** testes verdes, README, LICENSE e CHANGELOG presentes, e nenhuma dependência de framework em tempo de execução no núcleo. **Certificação:** revisão por dois integrantes e build verde na integração contínua. **Classificação:** por tipo, por domínio e por maturidade (experimental, estável, descontinuado). **Descontinuidade:** anunciada com uma versão MINOR de antecedência, com `@Deprecated`, e remoção só na MAJOR seguinte. **Avaliação:** semestral, com uso, defeitos e custo de manutenção	`docs/ESTRATEGIA_DE_ATIVOS.md`	O documento existe e cada ativo do catálogo cita a classificação e o critério de aceitação que cumpriu
GRU 2	**Publicar** o `smartscale-core` num repositório remoto de artefatos e manter um	GitHub Packages (registro Maven) com `distributionManagement` no `pom.xml` e `mvn deploy`;	Um computador sem o código-fonte baixa o JAR declarando só a

Resultado	Proposta	Artefato ou ferramenta	Como saber que atende
	catálogo com a localização de cada ativo	tag `smartscale-core-v1.0.0`; `docs/CATALOGO_DE_ATIVOS.md`	dependência e o repositório
GRU 3	**Registrar o uso**: a cada produto que adota ou troca a versão de um ativo, uma linha no catálogo (produto, ativo, versão, data, responsável)	Tabela "Quem usa" em `docs/CATALOGO_DE_ATIVOS.md`; estatística de downloads do GitHub Packages	O catálogo responde "quem seria afetado se a 1.0.0 mudar?"
GRU 4	**Manutenção periódica**: revisão mensal com checklist (dependências, issues abertas, testes, documentação). **Controle de mudança**: toda alteração do núcleo entra por pull request com integração contínua e linha no CHANGELOG	GitHub Actions rodando `mvn -pl smartscale-core verify` em cada pull request; modelo de pull request com o item "atualizei o CHANGELOG"	Cada versão no CHANGELOG corresponde a um pull request com build verde
GRU 5	**Notificar os usuários**: cada versão vira um GitHub Release com as notas da versão (quem acompanha o repositório é avisado); defeitos conhecidos ficam em issues com o rótulo do ativo; descontinuidade é avisada pelo `@Deprecated` (o compilador alerta quem usa) e nas notas da versão; os produtos do registro da GRU 3 são avisados diretamente	GitHub Releases, GitHub Issues, `@Deprecated`	Cada versão publicada tem um Release, e cada produto do registro recebeu o aviso

2. Desenvolvimento para Reutilização (DRU)

> *Propósito: identificar oportunidades de reutilização sistemática na organização e, se possível, estabelecer um programa de reutilização para desenvolver ativos a partir da engenharia de domínios de aplicação.*

2.1 Classificação

Resultado esperado	Situação	Justificativa (evidência no projeto)
DRU 1. Domínios de aplicação com oportunidades de reutilização são identificados, com o respectivo potencial	**Verde**	O domínio "escalas de serviço por rodízio" está identificado em `docs/VARIABILIDADE.md` e no modelo de características, com produtos-alvo (militar, hospitalar, vigilância) e o que é comum a todos (o núcleo). O potencial foi

Resultado esperado	Situação	Justificativa (evidência no projeto)
		demonstrado pelo teste `MotorDeRodizioTest.outroProdutoDaLinha_usaOutroCriterioSemMudarOMotor`
DRU 2. A capacidade de reutilização sistemática da organização é avaliada e ações corretivas são tomadas	**Vermelho**	A equipe não fez uma avaliação formal da própria capacidade de reutilização (ferramentas, conhecimento, processo)
DRU 3. Um programa de reutilização, com propósitos, escopo, metas e objetivos, é planejado	**Amarelo**	Os planos de implementação (em especial o Plano 5) definem objetivos de reuso: extrair o núcleo, empacotá-lo e versioná-lo. Mas são planos de projeto, não um programa de reutilização com metas para vários produtos
DRU 4. O programa de reutilização é implantado, monitorado e avaliado	**Amarelo**	O que foi planejado foi implantado: o núcleo existe, é empacotado e o MilScale o consome como dependência. Não há monitoramento nem avaliação dos resultados do reuso
DRU 5. Propostas de reutilização são avaliadas para garantir que o resultado seja apropriado para a aplicação alvo	**Vermelho**	As decisões de reuso foram tomadas pela equipe (por exemplo, deixar elegibilidade fora do núcleo), mas sem um procedimento ou registro de avaliação de propostas
DRU 6. Formas de representação para modelos e arquiteturas de domínio são selecionadas	**Verde**	**Modelo de domínio:** modelo de características no FeatureIDE (`docs/modelo-de-caracteristicas/`). **Arquitetura:** arquitetura hexagonal (domínio, aplicação, adaptadores) com regras verificadas por testes ArchUnit, e a separação núcleo × especialização × adaptadores descrita em `docs/VARIABILIDADE.md`
DRU 7. Um modelo de domínio é desenvolvido, com limites e relações com outros domínios estabelecidos e mantidos, capaz de capturar características comuns, variantes, opcionais e obrigatórias	**Verde**	O modelo de características da SmartScale (73 features, 12 restrições) captura o comum (obrigatórias), o variante (grupos Or e Alternative) e o opcional, e o documento do modelo registra os limites do domínio e a relação com RH, controle de ponto e autenticação corporativa. Está versionado com o código e é validado por script
DRU 8. Uma arquitetura de domínio, descrevendo uma família de aplicações, é desenvolvida e mantida	**Amarelo**	A arquitetura existe e é verificada: núcleo sem Spring nem JPA, produto especializando os contratos, adaptadores separados, ArchUnit. O que falta é ela estar descrita **para a**

Resultado esperado	Situação	Justificativa (evidência no projeto)
		família (como nasce um novo produto, onde ficam os pontos de variação, quando cada um é decidido) num documento próprio, com diagrama
DRU 9. Ativos do domínio são especificados, adquiridos ou desenvolvidos, e mantidos	**Verde**	O `smartscale-core` é especificado (API pública no README), desenvolvido (módulo Maven com 7 testes) e mantido (versão 1.0.0, CHANGELOG, SemVer). Os templates do gerador Database First (`database-first/templates/`) são um segundo ativo, de geração de código

2.2 Como adequar o protótipo aos resultados DRU 6, 7, 8 e 9

Resultado	Situação	Proposta	Artefato ou ferramenta
DRU 6	**Verde**	Manter as representações escolhidas e registrar a decisão: FeatureIDE para o modelo de domínio e diagrama de componentes UML para a arquitetura	Seção "Representações adotadas" em `docs/ARQUITETURA_DE_DOMINIO.md`
DRU 7	**Verde**	**Manter o modelo vivo:** todo pull request que cria ou remove uma feature atualiza o `model.xml`, e a integração contínua roda o `validar_modelo.py`, que reprova modelo com feature morta, falsamente opcional ou restrição redundante	GitHub Actions; `docs/modelo-de-caracteristicas/validar_modelo.py`
DRU 8	**Amarelo**	**Escrever a arquitetura de domínio:** as três camadas da família (núcleo, especialização, aplicação e adaptadores), os pontos de variação com o momento de decisão (critério da fila na inicialização, persistência pelo perfil do Spring, implantação pelo Compose), as regras ArchUnit que a protegem e o passo a passo de um produto novo (Plantão Hospitalar), com diagrama de componentes	`docs/ARQUITETURA_DE_DOMINIO.md`
DRU 9	**Verde**	**Ampliar e catalogar os ativos:** registrar no catálogo da GRU o núcleo, o modelo de características e os templates do Database First, cada um com versão e responsável; publicar o núcleo num repositório remoto (proposta da GRU 2)	`docs/CATALOGO_DE_ATIVOS.md`; GitHub Packages

Observação sobre DRU 6 e 7: antes deste trabalho, o modelo de domínio era só uma árvore em texto no `docs/VARIABILIDADE.md`, sem restrições nem validação, o que daria Amarelo nos dois. O modelo de características no FeatureIDE é o que os coloca em Verde.

3. Resumo

Processo	Verde	Amarelo	Vermelho
GRU (5 resultados)	0	4 (GRU 1, 2, 4, 5)	1 (GRU 3)
DRU (9 resultados)	4 (DRU 1, 6, 7, 9)	3 (DRU 3, 4, 8)	2 (DRU 2, 5)

O projeto é mais forte em **desenvolver para reutilização** (DRU): ele tem domínio, modelo, arquitetura e ativo. É mais fraco em **gerenciar o ciclo de vida** dos ativos (GRU). Isso é esperado num projeto acadêmico com um único produto. As propostas das tabelas 1.2 e 2.2 levam a GRU para Verde sem mudar o código do núcleo: elas são de processo, publicação e documentação.